

KanooniBaat — Complete Platform Blueprint

Full Architecture, Feature Specification & Agentic Build Prompts

Version 1.0 | Production-Grade | Monorepo Architecture

Built for: Turborepo + pnpm + Next.js + NestJS + Expo + PostgreSQL + AI

TABLE OF CONTENTS

1. Vision & Product Philosophy
 2. Complete Feature Specification
 3. Tech Architecture Decision
 4. Monorepo Structure
 5. Database Schema Design
 6. Phase-wise Implementation Plan
 7. Website — Prompt to Build
 8. Mobile App — Prompt to Build
 9. Backend API — Prompt to Build
 10. Shared Packages — Prompt to Build
 11. AI Services — Prompt to Build
 12. Infrastructure & DevOps
-

1. VISION & PRODUCT PHILOSOPHY {#vision}

The North Star

KanooniBaat = India's legal infrastructure layer. Not just an app. A platform that becomes the word people say when they need legal help — like "Google it" or "Practo kar lo."

Two Users, One Platform

- **Naagriks (Common People):** Scared, confused, need guidance fast. Treat them like a worried family member, not a customer.
- **Vakils (Lawyers/Advocates):** Overworked, under-tooled, want more clients and less admin. Make them feel powerful, not replaced.

Non-Negotiables (Every Feature Must Respect These)

1. **Privacy First:** Platform owner cannot access user data. End-to-end encryption for documents. DPDP Act compliance mandatory from day 0.
 2. **AI Guides, Lawyers Decide:** AI gives information and direction. All advice paths to a verified human lawyer. Never autonomous legal advice.
 3. **Vernacular by Default:** Every user-facing string must support Hindi + 11 regional languages. Not a feature — a foundation.
 4. **Mobile-First:** 80%+ of India accesses internet via phone. Web is trust/discovery. App is the product.
 5. **Lawyer Dignity:** Never frame lawyers as commodities. They are professionals being empowered, not replaced or rated like Uber drivers.
-

2. COMPLETE FEATURE SPECIFICATION {#features}

MODULE 1: AUTHENTICATION & IDENTITY

1.1 Multi-role Authentication

- Roles: **NAAGRIK** (common user), **VAKIL** (lawyer), **ADMIN**, **SUPER_ADMIN**
- Auth methods: Phone OTP (primary), Email OTP, Google OAuth
- Aadhaar-based e-KYC for lawyers (via DigiLocker API / Aadhaar XML)
- JWT access tokens (15 min) + Refresh tokens (30 days)
- Device fingerprinting for secure sessions
- Biometric login on mobile (Face ID / Fingerprint via Expo LocalAuthentication)

1.2 Lawyer Verification System

- Step 1: Phone + Email OTP
- Step 2: Bar Council Enrollment Number (validated against BCI records)
- Step 3: State Bar Council (dropdown — all 31 state bar councils)
- Step 4: Upload enrollment certificate (document + selfie with document)
- Step 5: Admin manual review + approval (SLA: 48 hours)
- Step 6: Verified badge issued
- Ongoing: Random re-verification checks

1.3 User Profile

Naagrik profile: Name, phone, email, location (district/state), preferred language, saved lawyers, case history, document vault access **Vakil profile:** Full name, Bar Council number, enrollment year, state, district, practice areas (multi-select taxonomy), languages spoken, consultation fee range, availability schedule, bio, education, experience, profile photo, office address (optional), UPI ID for payments

MODULE 2: NOTICE SCANNER (VIRAL ENTRY FEATURE)

2.1 How It Works

1. User uploads photo/PDF of any legal notice they received
2. OCR extracts text (Tesseract + Google Vision API fallback)
3. AI pipeline (Claude/GPT-4o) processes extracted text:
 - Identifies notice type (court summons, demand notice, rent notice, police notice, etc.)
 - Identifies issuing authority (court, lawyer firm, government dept, bank)
 - Flags if notice appears genuine or potentially fake (formatting checks, issuer verification)
 - Extracts: deadline dates, demanded amounts, case numbers, court names
 - Generates plain-language summary in user's preferred language
 - Generates "What you should do next" — 3-5 action steps
 - Recommends notice-specific lawyer category
4. Shareable result card (with KanooniBaat watermark) — the viral mechanic
5. CTA: "Talk to a lawyer about this notice" → lawyer matching flow

2.2 Free vs Premium

- Free: 2 scans/month, basic classification + plain-language summary
- Premium: Unlimited scans, response draft suggestion, precedent matches, priority lawyer connect

2.3 Notice Types Supported (Phase 1)

Legal notice from advocate, Court summons (civil/criminal), Demand notice (loan recovery), Rent/eviction notice, Employment termination letter, Consumer complaint notice, Income tax notice, GST notice, Police notice/FIR copy

MODULE 3: LEGAL EMERGENCY GUIDE ("KYA KAREIN?")

3.1 Scenario Library

Users select or describe their situation. AI + curated content gives:

- Immediate steps (what to do in next 1 hour)
- Rights in this situation
- Documents to gather
- What NOT to do
- When to call police vs lawyer
- Estimated timeline
- Connect to relevant lawyer

3.2 Core Scenarios (Phase 1 — 20 scenarios)

1. Received a legal notice
2. Got into a road accident
3. Arrested or detained by police
4. Domestic violence / harassment
5. Cheque bounce / financial fraud
6. Property / land dispute
7. Employer not paying salary
8. Consumer product/service fraud
9. Cyber crime / online fraud / sextortion
10. Eviction / landlord dispute
11. FIR filed against me
12. Marriage / divorce situation
13. Death in family — legal procedures
14. Workplace sexual harassment (POSH)
15. Bank account frozen
16. GST / Income tax raid
17. Child custody dispute
18. Loan recovery agent harassment
19. Property registration dispute
20. Will / inheritance dispute

3.3 AI + Curated Hybrid

- Each scenario has: human-curated base content reviewed by lawyer panel
 - AI personalises based on state (laws differ by state), situation details, and asks follow-up questions
 - All content tagged with: applicable law, relevant IPC/BNS section, jurisdiction
-

MODULE 4: LAWYER MARKETPLACE

4.1 Search & Discovery

Filters:

- Practice area (taxonomy of 40+ areas)
- Location (district, city, state, "near me")
- Language spoken
- Consultation type (chat, audio, video, in-person)
- Availability (today, this week)
- Fee range (₹0-₹200, ₹200-₹500, ₹500-₹1000, ₹1000+)
- Rating & review score
- Years of experience
- Case similarity match (AI-powered)

Sorting: Relevance (default AI match), Rating, Price (low/high), Experience, Distance

4.2 Lawyer Profile Page

- Profile photo + verified badge
- Bio (self-written)
- Practice areas with sub-specialisations
- Languages spoken
- Education (LLB college, year, additional degrees)
- Experience: Years of practice, courts practiced in
- Notable case types (without identifying details)
- Consultation options + fees for each
- Availability calendar

- Reviews & ratings (from verified clients only)
- Response time badge (avg. from platform data)

4.3 Consultation Booking

- Select consultation type (chat / audio / video)
- Select time slot (lawyer's availability calendar)
- Brief issue description (max 500 chars) → AI pre-categorises
- Payment (platform holds until consultation complete — escrow logic)
- Pre-consultation intake form auto-generated based on issue type
- Confirmation + calendar invite

4.4 Consultation Modes

In-App Chat:

- End-to-end encrypted messaging
- File/document sharing
- AI-assisted intake (before lawyer joins)
- Session timer
- Auto-save transcript (user's private copy)

Audio Call:

- WebRTC-based in-app call
- Call recording option (user consent + lawyer consent required)
- Fallback to phone call if connection poor

Video Call:

- WebRTC video
- Screen share (for document review)
- Recording with dual consent

4.5 Post-Consultation

- Rating + review (verified, after session completion)
- Follow-up scheduling
- Session summary auto-generated by AI (key points discussed)

- Lawyer's private notes (not visible to user)
 - Option to retain lawyer for ongoing representation
-

MODULE 5: LEGAL DOCUMENT VAULT (SAFEHOUSE)

5.1 Architecture

- Client-side encryption before upload (AES-256)
- Keys held by user (password-derived), NOT by platform
- Storage: AWS S3 with server-side encryption as second layer
- Access: Only user can decrypt. Platform sees only encrypted blobs.
- DPDP Act compliant. Privacy Charter publicly available.

5.2 Document Categories

- Property & Real Estate (sale deed, registry, khata, EC)
- Family Documents (marriage certificate, birth certificate, death certificate)
- Financial & Legal Agreements (loan agreements, mortgages, guarantees)
- Wills & Succession (will, nomination, succession certificate)
- Employment (offer letters, employment agreements, termination letters)
- Court Documents (FIRs, summons, judgments, orders)
- Identity Documents (Aadhaar, PAN, passport, driving license)
- Rental (rent agreement, notice, eviction letters)
- Business (partnership deed, company registration, GST certificate)
- Insurance (policy documents, claim letters)

5.3 Features

- Folder organisation (user-defined)
- Document tagging
- Expiry date tracking (alert before document expires)
- AI-powered content extraction: "What does this document say?" summary
- Contextual insight: "Based on your rent agreement, here's what applies if your landlord gives you an eviction notice"
- Share with lawyer (time-limited, revocable access link)
- QR-code verified document sharing (for third parties)

- Version history (original + annotated versions)

5.4 Free vs Premium

- Free: 5 documents, 50MB storage
 - Premium Individual: Unlimited documents, 5GB, AI insights, sharing
 - Family Plan: 5 family members, 10GB
-

MODULE 6: LAWYER — CASE MANAGEMENT SUITE

6.1 Case Lifecycle

Every case moves through states:

INTAKE → ACTIVE → HEARING_SCHEDULED → PENDING_DOCS → JUDGEMENT → CLOSED | APPEALED

Each case has:

- Case ID + court case number
- Client details (linked to platform user or offline client)
- Case type + court + jurisdiction
- Opposing party details
- Assigned lawyer + co-counsel
- Linked documents
- Hearing history (date, what happened, next date)
- Notes (lawyer-private + client-shared)
- Task list with deadlines
- Billing linked to case

6.2 Hearing Tracker

- Add hearing dates with court name, courtroom, judge name
- Pre-hearing reminder (3 days, 1 day, 1 hour before)
- Post-hearing quick update: what happened, next date, action items
- Integration with NJDG (National Judicial Data Grid) for auto-status fetch

6.3 Document Management per Case

- Upload documents, organise by type
- OCR + AI extraction: auto-fill key facts from uploaded documents
- Version control
- E-signature integration (for documents requiring signature)
- Secure sharing with client (lawyer controls access)

6.4 Task & Deadline Manager

- Case-linked tasks with due dates
 - Priority flags
 - Assign to self or junior
 - Automated task suggestions based on case type ("After hearing, always draft next 7 things")
 - Calendar view of all tasks across all cases
-

MODULE 7: AI LEGAL RESEARCH ENGINE (VAKIL AI)

7.1 Capabilities

- Search across 4M+ Indian court judgments (SCC Online, Manupatra, Indian Kanoon data)
- Natural language query: "Find cases where section 138 NI Act cheque bounce was compounded after 7 years"
- Filter: Court (SC/HC/District), Year range, Jurisdiction, Judge name, Case outcome
- Instant judgment summary (1-paragraph AI summary)
- Key ratio decidendi extraction
- "Cases like this" similarity search
- Citation chain builder: "All cases that cite Arnesh Kumar v. State of Bihar"
- Download in formatted citation style (Bluebook, SCC style)

7.2 Legal Drafting Assistant

- Template library (petition, reply, legal notice, affidavit, vakalatnama, bail application, etc.)
- AI fills template based on case facts (lawyer inputs key facts, AI drafts)
- Jurisdiction-aware (state-specific court formats)
- New BNS/BNSS/BSA 2023 section mapper: "This IPC section is now BNS section ___"
- Multi-language output (draft in Hindi + English)

- Track changes / version history of drafts

7.3 Law Library

- All central acts (searchable, section-by-section)
 - State amendment tracker
 - New law alerts (when Parliament passes relevant amendments)
 - Plain language version of every section (AI-generated + lawyer-reviewed)
-

MODULE 8: LAWYER — PRACTICE ANALYTICS & BILLING

8.1 Analytics Dashboard

- Total active cases, closed cases, pending hearings
- Win rate by case type (when outcome is logged)
- Revenue: monthly, quarterly, by case type
- Client acquisition: referral source, repeat clients
- Consultation to retention rate
- Average case duration by type
- Peer benchmarking (anonymised, opt-in)

8.2 Billing & Invoicing

- Create invoice against case or consultation
- Itemised billing: consultation fee, hearing fee, drafting fee, miscellaneous
- Auto-calculate from billable hours (timer on tasks)
- Send invoice via WhatsApp / email to client
- Payment collection via UPI, bank transfer
- Payment tracking (paid, pending, overdue)
- GST invoice generation

8.3 Client CRM

- Client contact list with case history
- Notes per client
- Follow-up reminders

- Client communication log (all in-app messages archived)
 - Referral tracking
-

MODULE 9: COURT CASE STATUS TRACKER (PUBLIC)

9.1 For Common Users

- Enter: Case number + court name OR CNR number
- Fetches from NJDG / eCourts API
- Shows: Case status, last hearing date, next hearing date, judge name, case stage
- Subscribe to case — get push notifications on any update
- Multiple cases (for different family members)

9.2 For Lawyers

- Bulk case tracking across all their active cases
 - Automated hearing date sync to lawyer's calendar
 - Alert if case marked "for orders" (critical status)
-

MODULE 10: COMMUNITY & CONTENT (TRUST BUILDER)

10.1 "Aapke Huqooq" Content Library

- Plain-language explanations of rights and laws
- Organised by life situation: Tenant, Employee, Consumer, Parent, Senior Citizen, Woman, Business Owner
- Available in 12 languages
- Searchable
- Short video format (for mobile) + long article format (for web)

10.2 Legal Q&A Forum

- Users post questions (anonymised)
- Verified lawyers answer (builds their profile + reputation)
- AI pre-answer with caveat to consult lawyer
- Upvoting, follow

- Lawyer can mark answers as paid consultations if detailed

10.3 Case Study Archive (Sanitised)

- Real case outcomes (names removed)
 - What happened, what law applied, what the outcome was
 - Search by situation type
 - "A case like mine" filter
-

3. TECH ARCHITECTURE DECISION {#architecture}

Final Stack (Justified)

```
MONOREPO TOOL:    Turborepo + pnpm workspaces
FRONTEND WEB:     Next.js 15 (App Router, RSC, Turbopack)
MOBILE:           Expo SDK 52 (React Native, bare workflow)
BACKEND:          NestJS 11 (modular, enterprise-grade)
API LAYER:        tRPC for internal (type-safe, zero boilerplate) + REST for
external/public APIs
DATABASE:         PostgreSQL 16 via Supabase (managed, RLS, realtime)
ORM:              Drizzle ORM (faster than Prisma, type-safe, direct SQL power)
CACHE:            Redis (Upstash for serverless, self-hosted for API)
SEARCH:           Meilisearch (lawyer search, judgment search – faster than
Elasticsearch for this scale)
AUTH:             Better Auth (open-source, cookie-based, replaces NextAuth)
FILE STORAGE:     AWS S3 + CloudFront CDN
REAL-TIME:        Supabase Realtime (for chat) + Socket.IO on NestJS
AI ORCHESTRATION: Vercel AI SDK (unified interface to Claude + OpenAI)
EMAIL:            Resend (developer-first, React Email templates)
SMS / OTP:        MSG91 (India-first, best delivery rates)
PAYMENTS:         Razorpay (India-first, UPI support, escrow APIs)
PUSH NOTIFS:      Expo Push Notifications + FCM
VIDEO/AUDIO CALL: LiveKit (open-source WebRTC, self-hostable)
OCR:              Google Cloud Vision API + Tesseract fallback
MONITORING:       Sentry (errors) + OpenTelemetry + Grafana
DEPLOYMENT:       Web → Vercel, API → Railway/Fly.io, Mobile → EAS Build
CI/CD:            GitHub Actions + Turborepo remote caching
```


Why These Choices Over Alternatives

Decision	Chosen	Over	Reason
ORM	Drizzle	Prisma	3-5x faster queries, type-safe SQL, no heavy client
API Style	tRPC (internal)	REST-only	Type-safety across web+mobile, no schema drift
Auth	Better Auth	NextAuth / Clerk	Open-source, self-hosted, custom flows for OTP
Search	Meilisearch	Elasticsearch	Faster setup, typo-tolerance, perfect for lawyer search
DB	Supabase PG	Neon/PlanetScale	RLS for per-user data isolation, Realtime for chat
Video	LiveKit	Twilio/Agora	Open-source, self-hostable, cost at scale
AI SDK	Vercel AI SDK	LangChain	Lighter, streaming-first, unified Claude + GPT interface
Monorepo	Turborepo	Nx	Simpler config, better Next.js integration, remote cache

4. MONOREPO STRUCTURE {#structure}

```
kanooni-baat/
├─ apps/
│   └─ web/                                # Next.js 15 (marketing + web app)
│       └─ app/
│           ├── (marketing)/                # Landing, about, features, blog
│           ├── (auth)/                    # Login, register, verify
│           ├── (naagrik)/                  # User dashboard, vault, cases
│           ├── (vakil)/                   # Lawyer dashboard, case mgmt
│           ├── (admin)/                   # Admin panel
│           └─ api/                        # Next.js API routes (webhooks, etc.)
│               ├── components/            # Web-specific components
│               ├── lib/                   # Web utilities
│               └─ public/                 # Static assets
│
└─ api/                                    # NestJS backend
    └─ src/
        └─ modules/
```

```

├── auth/
├── users/
├── lawyers/
├── consultations/
├── cases/
├── documents/
├── notices/           # Notice scanner
├── legal-guide/       # Emergency guide
├── search/            # Meilisearch integration
├── ai/                # AI orchestration
├── payments/
├── notifications/
├── realtime/          # WebSocket gateway
├── content/           # CMS content
├── admin/
├── common/
├── guards/
├── decorators/
├── filters/
├── interceptors/
├── middleware/
├── database/
├── schema/            # Drizzle schema files
├── migrations/
├── config/
├── test/
├── mobile/            # Expo React Native
├── app/               # Expo Router (file-based routing)
├── (auth)/
├── (naagrik)/
├── index.tsx          # Home
├── scan/              # Notice scanner
├── guide/             # Emergency guide
├── lawyers/           # Marketplace
├── vault/             # Document vault
├── cases/             # Case tracker
├── profile/
├── (vakil)/
├── index.tsx          # Lawyer home
├── cases/             # Case management
├── research/          # AI research
├── clients/           # CRM
├── schedule/          # Calendar

```

```

|   |   |   └─ analytics/
|   |   └─ consultation/      # Shared chat/call screens
|   └─ components/           # Mobile-specific components
|   └─ hooks/
|   └─ lib/
|
└─ packages/
|   └─ ui/                    # Shared design system
|       └─ src/
|           └─ components/    # Button, Input, Card, Modal, etc.
|           └─ tokens/        # Colors, spacing, typography
|               └─ index.ts
|           └─ package.json
|
|   └─ api-client/           # tRPC client + React Query setup
|       └─ src/
|           └─ trpc/
|               └─ hooks/     # useUser, useLawyers, etc.
|           └─ package.json
|
|   └─ types/                 # Shared TypeScript types + Zod schemas
|       └─ src/
|           └─ auth.ts
|           └─ user.ts
|           └─ lawyer.ts
|           └─ consultation.ts
|           └─ case.ts
|           └─ document.ts
|               └─ index.ts
|           └─ package.json
|
|   └─ config/                # Shared configs
|       └─ eslint/
|       └─ typescript/
|       └─ tailwind/
|       └─ prettier/
|
|   └─ utils/                 # Shared utility functions
|       └─ src/
|           └─ validation/     # Zod schemas (form + API)
|           └─ formatting/     # Date, currency, name formatters
|           └─ constants/      # Practice areas, states, courts
|               └─ i18n/       # Translation keys
|           └─ package.json

```

```
|
├─ turbo.json
├─ pnpm-workspace.yaml
├─ package.json
└─ .env.example
```

5. DATABASE SCHEMA DESIGN {#database}

Core Tables (Drizzle Schema)

typescript

```

// users table
users {
  id: uuid PK
  phone: string UNIQUE
  email: string UNIQUE nullable
  role: enum('NAAGRIK', 'VAKIL', 'ADMIN')
  preferred_language: enum('hi', 'en', 'ta', 'te', 'kn', 'mr', 'gu', 'bn', 'pa', 'ml', 'or', 'a
  is_verified: boolean
  is_active: boolean
  created_at: timestamp
  updated_at: timestamp
}

// naagrik_profiles table
naagrik_profiles {
  id: uuid PK
  user_id: uuid FK → users
  full_name: string
  state: string
  district: string
  avatar_url: string nullable
}

// vakil_profiles table
vakil_profiles {
  id: uuid PK
  user_id: uuid FK → users
  full_name: string
  bar_council_number: string UNIQUE
  state_bar_council: string
  enrollment_year: integer
  practice_areas: text[] (array)
  languages: text[]
  experience_years: integer
  bio: text
  office_address: jsonb
  consultation_fees: jsonb // { chat: 200, audio: 500, video: 1000 }
  availability: jsonb // weekly schedule
  rating: decimal
  total_reviews: integer
  is_approved: boolean
  approved_at: timestamp
  verification_docs: jsonb

```

```
}

// consultations table
consultations {
  id: uuid PK
  naagrik_id: uuid FK → users
  vakil_id: uuid FK → users
  type: enum('CHAT', 'AUDIO', 'VIDEO')
  status: enum('PENDING', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'DISPUTED')
  issue_summary: text
  issue_category: string
  scheduled_at: timestamp
  started_at: timestamp nullable
  ended_at: timestamp nullable
  duration_minutes: integer nullable
  fee_amount: decimal
  payment_id: string nullable
  payment_status: enum('PENDING', 'HELD', 'RELEASED', 'REFUNDED')
  rating: integer nullable // 1-5
  review: text nullable
  created_at: timestamp
}

// messages table (for chat consultations)
messages {
  id: uuid PK
  consultation_id: uuid FK → consultations
  sender_id: uuid FK → users
  content: text // encrypted at rest
  content_type: enum('TEXT', 'FILE', 'IMAGE', 'AUDIO')
  file_url: string nullable
  is_read: boolean
  created_at: timestamp
}

// cases table (lawyer case management)
cases {
  id: uuid PK
  vakil_id: uuid FK → users
  client_id: uuid FK → users nullable // null if offline client
  client_name: string // for offline clients
  client_phone: string nullable
  case_number: string nullable // court case number
  cnr_number: string nullable // court CNR
}
```

```

    court_name: string
    court_type: enum('DISTRICT', 'HIGH_COURT', 'SUPREME_COURT', 'TRIBUNAL', 'OTHER')
    state: string
    district: string
    case_type: string
    description: text
    status: enum('INTAKE', 'ACTIVE', 'HEARING_SCHEDULED', 'PENDING_DOCS', 'JUDGEMENT', 'CLOSED')
    opposing_party: string nullable
    next_hearing_date: date nullable
    fee_agreed: decimal nullable
    created_at: timestamp
    updated_at: timestamp
    closed_at: timestamp nullable
    outcome: text nullable
}

// case_hearings table
case_hearings {
    id: uuid PK
    case_id: uuid FK → cases
    hearing_date: date
    court_room: string nullable
    judge_name: string nullable
    what_happened: text nullable
    next_date: date nullable
    action_items: text[]
    created_at: timestamp
}

// case_documents table
case_documents {
    id: uuid PK
    case_id: uuid FK → cases
    uploaded_by: uuid FK → users
    file_name: string
    file_type: string
    s3_key: string
    file_size: integer
    is_visible_to_client: boolean default false
    created_at: timestamp
}

// vault_documents table (user's personal document vault)
vault_documents {

```

```
id: uuid PK
user_id: uuid FK → users
category: string
file_name: string
encrypted_s3_key: string // only encrypted blob stored
encrypted_metadata: text // encrypted JSON with AI-extracted metadata
tags: text[]
expiry_date: date nullable
file_size: integer
created_at: timestamp
updated_at: timestamp
}
```

// notice_scans table

```
notice_scans {
  id: uuid PK
  user_id: uuid FK → users nullable // null for anonymous
  original_file_key: string
  extracted_text: text
  notice_type: string
  issuing_authority: string nullable
  deadline_date: date nullable
  is_likely_genuine: boolean nullable
  ai_summary: text // in user's language
  recommended_actions: text[]
  recommended_lawyer_category: string
  language: string
  created_at: timestamp
}
```

// lawyer_cases (tracking feature for users)

```
tracked_cases {
  id: uuid PK
  user_id: uuid FK → users
  cnr_number: string
  court_name: string
  case_title: string
  last_fetched_at: timestamp
  last_status: text
  notifications_enabled: boolean
}
```

// payments table

```
payments {
```



```

id: uuid PK
consultation_id: uuid FK → consultations nullable
payer_id: uuid FK → users
payee_id: uuid FK → users
amount: decimal
currency: string default 'INR'
razorpay_order_id: string nullable
razorpay_payment_id: string nullable
status: enum('CREATED', 'CAPTURED', 'HELD', 'RELEASED', 'REFUNDED', 'FAILED')
created_at: timestamp
}

// subscriptions table
subscriptions {
  id: uuid PK
  user_id: uuid FK → users
  plan: enum('FREE', 'NAAGRIK_PRO', 'VAKIL_BASIC', 'VAKIL_PRO', 'VAKIL_PREMIUM', 'SME')
  status: enum('ACTIVE', 'CANCELLED', 'EXPIRED', 'PAST_DUE')
  current_period_start: timestamp
  current_period_end: timestamp
  razorpay_subscription_id: string nullable
  created_at: timestamp
}

// reviews table
reviews {
  id: uuid PK
  consultation_id: uuid FK → consultations
  reviewer_id: uuid FK → users
  reviewed_id: uuid FK → users
  rating: integer // 1-5
  review: text nullable
  is_visible: boolean default true
  created_at: timestamp
}

// notifications table
notifications {
  id: uuid PK
  user_id: uuid FK → users
  type: string // 'HEARING_REMINDER', 'CONSULTATION_CONFIRMED', etc.
  title: string
  body: string
  data: jsonb

```

```
  is_read: boolean
  created_at: timestamp
}

// content table (Aapke Huqooq articles)
content_articles {
  id: uuid PK
  slug: string UNIQUE
  category: string
  title: jsonb // { en: '...', hi: '...' }
  body: jsonb // { en: '...', hi: '...' }
  tags: text[]
  life_situation: string
  applicable_laws: text[]
  reviewed_by_vakil_id: uuid FK → users nullable
  published_at: timestamp nullable
  is_published: boolean
  views: integer default 0
}
```

6. PHASE-WISE IMPLEMENTATION PLAN {#phases}

Phase 0: Foundation (Week 1-2)

Goal: Monorepo initialized, all tools configured, CI/CD running, environments set up

Tasks:

- ☐ Initialize Turborepo + pnpm workspace
- ☐ Create all apps and packages directories
- ☐ Configure TypeScript (strict mode, path aliases)
- ☐ Configure ESLint + Prettier (shared from packages/config)
- ☐ Set up Tailwind CSS (shared config)
- ☐ Set up Drizzle + Supabase connection
- ☐ Set up Better Auth
- ☐ Configure GitHub Actions (lint, type-check, build)
- ☐ Set up Sentry (error monitoring)
- ☐ Environment variables template (.env.example)
- ☐ Set up Vercel project (web) + Railway project (api)
- ☐ EAS Build setup (mobile)

Phase 1: Website — Trust & Visibility (Week 2-4)

Goal: Public-facing website live. World can see KanooniBaat, understand the product, waitlist sign-up.

Features to build:

- Landing page (hero, features overview, how it works, stats, testimonial placeholders)
 - Features page (detailed feature descriptions)
 - For Lawyers page (lawyer-specific pitch)
 - Pricing page (subscription tiers)
 - About page (vision, team)
 - Blog (content marketing — powered by MDX)
 - Lawyer early-access waitlist form
 - User app waitlist form
 - Legal pages (Privacy Policy, Terms of Service, Privacy Charter)
 - SEO setup (metadata, sitemap, OG images)
-

Phase 2: Auth + Core API (Week 3-5)

Goal: Authentication working across web and mobile. Core database running.

Features to build:

- Phone OTP auth (MSG91 integration)
 - Email OTP auth
 - JWT + Refresh token strategy
 - Role-based guards (NestJS)
 - User profile CRUD
 - Lawyer profile creation
 - Admin panel (basic — user list, lawyer approval queue)
 - Database migrations (all core tables)
 - API health checks + Swagger docs
-

Phase 3: Lawyer Verification + Marketplace (Week 4-7)

Goal: Lawyers can sign up, get verified, be discovered.

Features to build:

- Lawyer registration multi-step flow
 - Bar Council number capture + admin verification workflow
 - Lawyer profile page (public)
 - Lawyer search (Meilisearch integration, all filters)
 - Lawyer discovery on web + mobile
 - Lawyer availability scheduling
-

Phase 4: Notice Scanner (Week 5-7)

Goal: Viral entry feature live. Anyone can scan a notice.

Features to build:

- File upload (PDF, image)
 - OCR pipeline (Google Vision API)
 - AI analysis pipeline (Vercel AI SDK → Claude)
 - Result display (type, summary, actions, genuineness flag)
 - Shareable result card (OG image generation)
 - Free tier limits enforcement
-

Phase 5: Consultation Flow + Payments (Week 6-9)

Goal: Users can book and complete consultations with lawyers. Money flows.

Features to build:

- Consultation booking (select lawyer → type → slot → issue)
- Razorpay integration (order creation, payment capture, webhook handling)
- Payment escrow logic (hold → release on completion)
- In-app chat (Supabase Realtime)
- Audio/Video calls (LiveKit integration)
- Post-consultation flow (rating, review, summary)

- Lawyer payout (Razorpay X)
-

Phase 6: Legal Emergency Guide (Week 7-8)

Goal: "Kya Karein?" feature live for 20 core scenarios.

Features to build:

- Scenario taxonomy + selection UI (mobile-first)
 - AI personalisation pipeline (state + situation → tailored guide)
 - Content management for scenario base content
 - Lawyer connect CTA at end of guide
 - Content in Hindi + English (Phase 1)
-

Phase 7: Document Vault (Week 8-11)

Goal: Users can securely store their lifetime legal documents.

Features to build:

- Client-side encryption (Web Crypto API / React Native Crypto)
 - S3 upload with encryption
 - Folder + tag organisation
 - Expiry date tracking + alerts
 - AI document summary (on-demand, encrypted → decrypt → summarise → discard)
 - Share with lawyer (time-limited link)
 - Free tier limits
-

Phase 8: Lawyer Case Management Suite (Week 9-13)

Goal: Lawyers have a complete practice management system.

Features to build:

- Case creation + all metadata
- Case status lifecycle
- Hearing tracker + calendar

- Task & deadline manager
 - Case document management
 - Client CRM
 - NJDG integration (court case status fetch)
-

Phase 9: AI Legal Research Engine (Week 11-15)

Goal: Lawyers have AI-powered research at their fingertips.

Features to build:

- Judgment search (Meilisearch, indexed judgment corpus)
 - Natural language query interface
 - Judgment summary AI pipeline
 - Citation chain builder
 - BNS/BNSS section mapper
 - AI drafting assistant (template + AI fill)
 - Law library (all central acts)
-

Phase 10: Practice Analytics + Billing (Week 13-16)

Goal: Lawyers have full business intelligence.

Features to build:

- Analytics dashboard (cases, revenue, win rate, client acquisition)
 - Billable hour timer (linked to tasks)
 - Invoice creation + PDF generation
 - Payment collection (UPI QR via Razorpay)
 - GST invoice support
 - Revenue reports (monthly/quarterly export)
-

Phase 11: Push Notifications + Case Tracker (Week 12-14)

Goal: Users get proactive alerts for hearings, messages, case updates.

Features to build:

- Expo Push Notifications setup
 - FCM integration
 - Hearing reminders (3 day, 1 day, 1 hour)
 - NJDG case status tracker (public feature)
 - Message notifications
 - Consultation reminders
-

Phase 12: Content Platform + Q&A (Week 14-17)

Goal: Trust-building content live. Community Q&A running.

Features to build:

- "Aapke Huqooq" article library (Hindi + English, 12 languages roadmap)
 - Search within content
 - Legal Q&A forum
 - Lawyer answer system
 - AI pre-answer with lawyer consult CTA
-

Phase 13: Subscriptions + Monetisation (Week 15-18)

Goal: Full subscription system running. Platform generating sustainable revenue.

Features to build:

- Subscription plan management (Razorpay Subscriptions)
 - Plan limits enforcement (feature gates)
 - Upgrade/downgrade flows
 - Billing history
 - Invoice emails
 - Analytics: MRR, churn, LTV
-

Phase 14: i18n + Vernacular (Week 16-19)

Goal: Platform usable in 8 languages natively.

Features to build:

- i18n setup (next-intl for web, i18n-js for mobile)
 - Translation pipeline (React + AI-assisted translations)
 - Language switcher (web + mobile)
 - RTL support (Urdu/Arabic if needed)
 - Language preference per-user
 - Priority languages: Hindi, English, Tamil, Telugu, Kannada, Marathi, Gujarati, Bengali
-

Phase 15: WhatsApp Bot + Integrations (Week 18-22)

Goal: WhatsApp as acquisition channel. External integrations live.

Features to build:

- WhatsApp Business API (via 360dialog or Twilio)
 - Bot flows: notice scanner, emergency guide, lawyer connect
 - Deep link back to app
 - DigiLocker integration (document fetch)
 - Bar Council API integration (when available)
-

7. WEBSITE BUILD PROMPT {#website-prompt}

PROMPT: KanooniBaat Website – Next.js 15 App Router

You are building the public-facing website for KanooniBaat, India's premier legal technology platform. This is a Next.js 15 application using App Router, TypeScript (strict), Tailwind CSS, and Framer Motion.

CONTEXT:

KanooniBaat is positioning itself as "Practo for legal services" in India – a platform that demystifies legal services for common people while giving lawyers powerful tools to manage their practice. The brand is trustworthy, approachable, Indian, and professional. Think clean, modern, and calm – not aggressive or salesy.

PRIMARY COLORS: Deep navy (#0F172A), Legal green (#16A34A), Warm amber (#F59E0B), Pure white

TYPOGRAPHY: Geist Sans (headings), Geist (body) – Next.js native
LANGUAGES: English (primary), Hindi (secondary) – i18n setup required

ROUTE STRUCTURE (App Router):

```
app/
├── (marketing)/
│   ├── page.tsx                # Landing page
│   ├── features/page.tsx       # All features
│   ├── for-lawyers/page.tsx    # Lawyer-specific page
│   ├── pricing/page.tsx        # Pricing plans
│   ├── about/page.tsx          # Mission, team, vision
│   ├── blog/
│   │   ├── page.tsx            # Blog listing
│   │   └── [slug]/page.tsx     # Blog post (MDX)
│   ├── privacy/page.tsx        # Privacy Policy
│   ├── terms/page.tsx          # Terms of Service
│   └── privacy-charter/page.tsx # Public privacy pledge
├── layout.tsx                  # Root layout with nav + footer
└── not-found.tsx
```

COMPONENTS TO BUILD:

1. NAVIGATION (Navbar)

- Logo: "KanooniBaat" in Devanagari + Latin
- Links: Features, For Lawyers, Pricing, About, Blog
- CTAs: "For Lawyers" (outlined), "Get App" (filled green)
- Mobile: hamburger menu with slide-in sheet
- Sticky on scroll, blur backdrop, border-bottom on scroll

2. LANDING PAGE SECTIONS (in order):

a) HERO SECTION

- Headline (Hindi + English alternating animation): "Kanooni Madad, Seedha Aapke Phone Par" / "Legal Help, Right in Your Pocket"
- Subheadline: "India's most trusted platform to find verified lawyers, scan legal notices, understand your rights, and manage your legal life – in your language."
- Two CTAs: "Scan a Notice (Free)" → opens a demo scanner, "Find a Lawyer" → pricing/waitlist
- Hero visual: floating phone mockup showing the app's home screen (SVG illustration)
- Trust bar below hero: "40M+ pending cases in India | 1.7M+ registered lawyers | Your legal partner is here"

b) PROBLEM STATEMENT SECTION

- "Legal India has a problem" – 3 stark stats cards
- Card 1: "40 Million cases pending in Indian courts"
- Card 2: "75% of Indians cannot afford a lawyer"
- Card 3: "Most people don't know their basic legal rights"
- Narrative: "The system is broken. KanooniBaat is not here to fix the courts – we're here to make sure you navigate them without fear."

c) HOW IT WORKS (3-step for each user type)

- Toggle: "I need legal help" / "I'm a lawyer"
- For users: 1. Describe your situation → 2. Get instant guidance → 3. Connect with verified lawyer
- For lawyers: 1. Create your profile → 2. Get verified → 3. Manage your practice with AI

d) CORE FEATURES GRID (6 features, icon + title + description)

- Notice Scanner, Emergency Legal Guide, Lawyer Marketplace, Document Vault, Case Tracker, Legal Library
- Each card: animated on scroll, click reveals full feature description

e) NOTICE SCANNER DEMO (Interactive)

- Live demo: upload an image OR paste text of a notice
- Shows: "What type is this?", plain language summary, recommended steps
- Powered by actual API call to backend
- CTA: "Try it – it's free"

f) FOR LAWYERS TEASER

- "For Vakils: Your practice, powered by AI"
- Feature highlights: Case management, AI research, billing, client discovery
- CTA: "Join as a Lawyer (Free for 3 months)"

g) TRUST SIGNALS

- "Privacy Charter" section – animated visual showing "Your data is yours"
- Bar Council verification badge explanation
- 3 lawyer testimonial cards (placeholder → real after launch)
- "Built with legal experts" advisor names (to be added)

h) PRICING PREVIEW

- Simple 3-column: Free, Pro (User), Vakil Pro
- Key features highlighted, not exhaustive
- CTA to full pricing page

i) APP DOWNLOAD SECTION

- "India's legal help, in your pocket"
- iOS + Android download buttons

- QR code for mobile
- Phone mockup (3D tilt on hover)

j) FAQ SECTION

- 8-10 common questions, accordion
- Questions: "Is my data safe?", "Are lawyers real/verified?", "What if I get wrong advice?", etc.

k) FOOTER

- Logo, tagline
- Links: Product, Company, Legal, Social
- WhatsApp contact
- Made in India IN badge

3. FOR LAWYERS PAGE

- Separate hero: "Grow your practice. Manage your cases. Research in seconds."
- Feature deep-dive: Lawyer profile, Case management, AI research, Billing, Client acquisition
- Pricing for lawyers (Vakil Basic/Pro/Premium)
- "How verification works" explainer
- Testimonials from lawyers
- FAQ for lawyers
- "Join as a Lawyer" CTA (form or waitlist)

4. PRICING PAGE

All plans in a comparison table:

- Naagrik Free: Notice scanner (2/mo), Emergency guide, Basic lawyer search, 5 documents vault
- Naagrik Pro (₹199/mo): Unlimited scanner, Priority matching, 5GB vault, AI insights, Case tracker
- Vakil Basic (₹499/mo): Profile + discovery, Chat consultations, 10 active cases
- Vakil Pro (₹1,499/mo): Everything + AI research, Case management, Billing, Analytics
- Vakil Premium (₹2,999/mo): Everything + API access, White-label consultation, Priority support
- SME (₹4,999/mo): Business legal package

5. TECHNICAL REQUIREMENTS:

- next-intl for i18n (en + hi routing)
- next-seo or Metadata API for SEO
- framer-motion for animations (scroll-triggered, page transitions)
- tailwindcss with custom design tokens
- shadcn/ui as base component library
- react-hook-form + zod for waitlist/contact forms

- Resend for form submissions
- next-mdx-remote for blog posts
- Automatic sitemap generation (next-sitemap)
- OG image generation (Next.js ImageResponse)
- Dark mode support (class-based, system preference)
- Lighthouse score target: 95+ on all metrics

6. SEO STRATEGY:

Target keywords: "legal help India", "find lawyer online India", "legal notice kya kare", "vakil dhundne ka app", "legal advice Hindi"

Meta structure, structured data (Law Firm, FAQ schema), sitemap, robots.txt

BUILD ORDER:

1. Layout (nav + footer) → 2. Landing page sections → 3. Pricing → 4. For Lawyers → 5. About → 6. Blog setup → 7. Legal pages → 8. i18n → 9. SEO → 10. Analytics (Vercel Analytics + PostHog)

8. MOBILE APP BUILD PROMPT {#mobile-prompt}

PROMPT: KanooniBaat Mobile App – Expo React Native (SDK 52)

You are building the core KanooniBaat mobile app – the actual product where users get legal help and lawyers manage their practice. This is an Expo SDK 52 app using Expo Router (file-based routing), TypeScript strict, NativeWind (Tailwind for RN), and React Query.

ARCHITECTURE:

- Expo Router v4 (file-based routing, like Next.js App Router for mobile)
- NativeWind v4 (Tailwind CSS for React Native)
- @tanstack/react-query v5 (data fetching + caching)
- tRPC client (type-safe API calls from shared packages/api-client)
- Zustand (local state: auth, user preferences)
- React Native Reanimated v3 (animations)
- Expo LocalAuthentication (biometrics)
- Expo Camera + ImagePicker (for notice scanning)
- Expo DocumentPicker (for vault)
- Expo Notifications (push notifications)
- Expo SecureStore (sensitive data – tokens, encryption keys)
- expo-crypto (client-side encryption for vault)
- LiveKit React Native SDK (video/audio calls)

APP STRUCTURE (Expo Router):

```
app/
├─ _layout.tsx          # Root: fonts, theme, query client, auth gate
├─ index.tsx            # Redirect based on auth/role
├─ (auth)/
│   ├─ _layout.tsx
│   ├─ welcome.tsx      # Splash/onboarding
│   ├─ phone.tsx        # Phone number entry
│   ├─ otp.tsx          # OTP verification
│   └─ role-select.tsx  # Naagrik or Vakil?
├─ (naagrik)/
│   ├─ _layout.tsx      # Bottom tab navigator
│   ├─ index.tsx        # Home (personalized dashboard)
│   ├─ scan/
│   │   ├─ index.tsx    # Notice scanner home
│   │   ├─ capture.tsx  # Camera capture
│   │   └─ result/[id].tsx # Scan result
│   ├─ guide/
│   │   ├─ index.tsx    # Emergency guide scenario list
│   │   └─ [scenario].tsx # Individual scenario guide
│   │       └─ chat.tsx  # AI chat for follow-up
│   ├─ lawyers/
│   │   ├─ index.tsx    # Search + browse lawyers
│   │   ├─ [id].tsx     # Lawyer profile
│   │   └─ book/[id].tsx # Booking flow
│   ├─ vault/
│   │   ├─ index.tsx    # Document vault home
│   │   ├─ upload.tsx   # Upload flow
│   │   └─ [id].tsx     # Document viewer
│   ├─ cases/
│   │   ├─ index.tsx    # My tracked cases
│   │   └─ [cnr].tsx    # Case status detail
│   ├─ consultation/
│   │   ├─ [id].tsx     # Active consultation
│   │   ├─ chat.tsx     # In-app chat
│   │   └─ call.tsx     # Audio/video call
│   └─ profile/
│       ├─ index.tsx    # User profile
│       ├─ settings.tsx # App settings
│       └─ subscription.tsx # Plan + billing
├─ (vakil)/
│   ├─ _layout.tsx      # Lawyer bottom tabs
│   └─ index.tsx        # Lawyer dashboard
```

```
| └─ cases/
|   └─ index.tsx          # Case list
|   └─ new.tsx           # New case intake
|   └─ [id]/
|     └─ index.tsx        # Case overview
|     └─ hearings.tsx    # Hearing log
|     └─ documents.tsx   # Case documents
|     └─ tasks.tsx       # Task list
|     └─ billing.tsx     # Case billing
| └─ research/
|   └─ index.tsx          # AI research home
|   └─ search.tsx        # Judgment search
|   └─ [judgmentId].tsx  # Judgment view
|   └─ draft.tsx         # AI drafting
| └─ clients/
|   └─ index.tsx          # Client CRM
|   └─ [id].tsx          # Client detail
| └─ schedule/
|   └─ index.tsx          # Calendar view (all hearings + consultations)
| └─ analytics/
|   └─ index.tsx          # Practice analytics
| └─ profile/
|   └─ index.tsx          # Lawyer profile editor
|   └─ subscription.tsx
```

SCREENS TO BUILD – DETAILED:

NAAGRIK HOME SCREEN (index.tsx)

Layout: ScrollView with pull-to-refresh

Top: Greeting + user name + language toggle

Quick Actions Row (4 large icons):

- Scan Notice (primary CTA, amber highlight)
- Find Lawyer (green)
- Kya Karein? (blue, emergency guide)
- My Documents (purple)

Active Consultation card (if any) – shows lawyer name, next step

Recent Scans (last 3 notice scans)

Featured Lawyers (based on location + past queries)

"Aapke Huqooq" content cards (2 relevant articles based on recent activity)

Tracked cases status (if any)

NOTICE SCANNER FLOW

Screen 1 – scan/index.tsx:

Header: "Notice Scanner" + info icon

Two options:

- A) "Take Photo of Notice" → camera capture
- B) "Upload File" → document picker (PDF/image)

Free tier counter: "2 of 2 free scans remaining"

Recent scans list

Screen 2 – scan/capture.tsx:

Expo Camera with:

- Document detection overlay (corner guides)
- Auto-capture on good document detection
- Manual capture button
- Flash toggle
- Gallery picker fallback

Processing overlay: animated AI spinner

Screen 3 – scan/result/[id].tsx:

Notice type badge (e.g., "Court Summons", "Demand Notice")

Genuineness indicator: "Likely Genuine ✓" / "Verify Carefully ⚠"

Plain-language summary (in user's language):

- What this notice means
- Who sent it
- Important dates
- Amount (if any)

"What you should do" – numbered step list

Recommended lawyer type

CTAs:

- "Talk to a Lawyer about this" → lawyer search pre-filtered
- "Save to Vault" → vault save flow
- "Share" → native share sheet (result card image)

EMERGENCY GUIDE – "KYA KAREIN?"

Screen 1 – guide/index.tsx:

Header: "Kya Karein? / What should you do?"

Search bar: "Describe your situation..."

Grid of situation cards (20 scenarios):

- Large icon + situation title (Hindi + English)
- Color coded by urgency (red=urgent, amber=serious, blue=informational)

Emergency hotlines bar at bottom (always visible):

- Police: 100, Women helpline: 1091, Legal Aid: 15100

Screen 2 – guide/[scenario].tsx:

Situation title

Urgency level badge

Personalisation form (collapsible):

- Your state (dropdown)
- Additional context (3-5 relevant questions, scenario-specific)

Guide content:

- Section 1: "Right Now (Next 1 hour)" – immediate steps
- Section 2: "Your Rights in This Situation" – key rights explained
- Section 3: "Documents to Gather" – checklist
- Section 4: "What NOT to Do" – common mistakes
- Section 5: "When to Involve Police / Court"
- Section 6: "Timeline – What to expect"

Applicable laws footer (tappable, shows plain-language explanation)

Bottom CTA: "Talk to a [relevant type] Lawyer" → lawyer marketplace with pre-filter

LAWYER MARKETPLACE

Screen 1 – lawyers/index.tsx:

Search bar + filter chips row (horizontally scrollable):

Active filters shown as removable chips

Filter bottom sheet (tap filter icon):

Practice Area (expandable taxonomy – Criminal, Civil, Family, Corporate, etc.)

Location (district + state, "Near me")

Language (multi-select)

Consultation type (Chat/Audio/Video)

Fee range slider

Availability (Today / This week)

Lawyer cards (list or grid toggle):

- Photo + verified badge
- Name + practice areas (top 2)
- Location + languages
- Rating + review count
- Consultation types available
- Lowest fee shown
- "Book" CTA

Featured Lawyers section at top (sponsored, clearly labeled)

"No results" state with suggested alternatives

Screen 2 – lawyers/[id].tsx (Lawyer Profile):

Header: Photo, name, verified badge, rating

Tab bar: Overview | Reviews | Schedule

Overview tab:

Practice areas (full list, tagged)

Languages spoken

Experience + education

Bio

Consultation options:

- Chat: ₹X/session
- Audio: ₹X/30min
- Video: ₹X/30min
- In-person: ₹X (optional)

Availability calendar (this week + next week, slots)

Book consultation CTA (sticky bottom)

Reviews tab:

Rating breakdown (5-star distribution)

Verified reviews only

Sorted by: Recent / Most helpful

Schedule tab:

Week view of available slots

Screen 3 – lawyers/book/[id].tsx (Booking Flow):

Step 1: Select consultation type (Chat/Audio/Video)

Step 2: Select date + time slot

Step 3: Describe your issue (textarea + category selector)

AI suggests: "Based on your description, this might be a [type] matter"

Step 4: Review + Pay

Fee breakdown, Razorpay payment sheet

Confirmation screen with:

Lawyer name, type, time

"Add to calendar" button

WhatsApp reminder option

IN-APP CONSULTATION SCREEN

consultation/chat.tsx:

Chat header: Lawyer name + photo + status indicator (online/offline)

Message list (bottom-to-top):

- Text bubbles (user right, lawyer left)
- File attachments (tap to preview)
- Timestamps
- Read receipts

Input bar:

- Text input
- Attachment (photo, file)
- Voice message (hold to record)
- Send button

"Consultation ends in X min" countdown if time-limited

"End consultation" button (after minimum time)

consultation/call.tsx (Audio/Video):

Full-screen video (video) or avatar with waveform (audio)

Controls overlay:

- Mute/unmute
- Camera on/off (video only)
- Speaker/earpiece toggle
- Chat sidebar toggle (to share docs during call)
- End call

Recording indicator (if consented)

Duration timer

DOCUMENT VAULT

vault/index.tsx:

Storage usage bar: "2.3 MB of 50 MB used (Free plan)"

Category tabs (horizontally scrollable):

All | Property | Family | Financial | Court | Identity | Employment | Rental |

Business

Document grid/list toggle

Document cards:

- Category icon + color
- Document name
- Upload date
- Expiry warning (if near expiry)

FAB: Add document

Expiring soon banner (if any docs expiring in 30 days)

vault/upload.tsx:

Step 1: Category selection (visual grid)

Step 2: Document source (Camera / Files / DigiLocker)

Step 3: Document details:

- Name (auto-suggested from AI)
- Tags (multi-select + custom)
- Expiry date (optional)

Step 4: Encryption notice + confirm upload

Progress indicator

VAKIL HOME SCREEN

Summary cards row:

- Active cases count
- Today's hearings
- Pending tasks
- This month's earnings

Today's agenda:

- Hearings (time + court + case name)
- Scheduled consultations (time + client name + type)

Urgent tasks (overdue or due today)

Pending consultation requests (unbooked inquiries)

Recent case activity feed

VAKIL CASE MANAGEMENT

cases/index.tsx:

Filter: All | Active | Hearing Today | Pending Docs | Closed

Sort: Next Hearing | Recent Activity | Client Name

Case cards:

- Client name + case type
- Court name + case number
- Status badge
- Next hearing date (highlighted if today/tomorrow)
- Days since last activity

Search bar

FAB: New case

cases/[id]/index.tsx (Case Overview):

Case header: Client name, status badge

Quick stats: Hearing count, documents count, tasks pending

Tab bar: Overview | Hearings | Documents | Tasks | Billing

Overview: All case metadata, opposing party, assigned court

cases/[id]/hearings.tsx:

Timeline of all hearings (reverse chronological)

Each entry: Date, what happened, action items, next date

FAB: Add hearing record

Upcoming hearing card (pinned top)

research/index.tsx:

Two tabs: "Search Judgments" | "Draft with AI"

Search tab:

- Natural language search input
- Filter chips: SC / HC / District | Year range | State
- Judgment cards (title, court, date, 1-line summary)
- Tap → full judgment view with AI summary + key holdings

Draft tab:

- Template picker (20+ templates)
- Case facts input form (template-specific fields)
- "Generate Draft" → streaming AI output
- Edit in-app → export as DOCX/PDF

DESIGN SYSTEM (NativeWind):

Primary: #16A34A (green-600)

Secondary: #0F172A (slate-900)

Accent: #F59E0B (amber-500)

Background: #FFFFFF (light) / #0F172A (dark)

Surface: #F8FAFC (light) / #1E293B (dark)

Error: #DC2626

Success: #16A34A

Typography:

- Heading: Inter Bold 24/20/18/16
- Body: Inter Regular 16/14
- Caption: Inter Regular 12
- Hindi: Noto Sans Devanagari (loaded via expo-font)

Spacing: 4px base unit (4, 8, 12, 16, 20, 24, 32, 40, 48)

Border radius: 8 (small), 12 (medium), 16 (large), 24 (xl)

Animations (Reanimated v3):

- Screen transitions: slide + fade
- List items: staggered fade-in on load
- Cards: spring scale on press
- Bottom sheets: spring slide-up
- Skeleton loading on all data screens

PERFORMANCE TARGETS:

- Cold start: < 2 seconds
- JS bundle: < 2MB (with code splitting)
- Image lazy loading everywhere
- Offline mode: Notice scanner results cached, vault accessible offline
- Optimistic updates on all mutations

9. BACKEND API BUILD PROMPT {#api-prompt}

PROMPT: KanooniBaat Backend API – NestJS 11

You are building the production-grade NestJS backend for KanooniBaat. This is the API powering both the Next.js web app and Expo mobile app.

CORE SETUP:

- NestJS 11 with modular architecture
- TypeScript strict mode
- Drizzle ORM + Supabase PostgreSQL
- tRPC server (for internal web/mobile API)
- REST endpoints (for webhooks, public API, external integrations)
- JWT auth with refresh tokens (Better Auth)
- Redis (Upstash) for caching + rate limiting
- Socket.IO for real-time (chat, notifications)
- Bull Queue (background jobs: AI processing, emails, NJDG sync)
- Swagger/OpenAPI documentation
- Zod validation on all inputs
- Global error handling + logging (Pino logger)
- Rate limiting (throttler)
- Helmet (security headers)
- CORS properly configured

MODULE STRUCTURE:

src/modules/

auth/

- POST /auth/send-otp (phone or email)
- POST /auth/verify-otp
- POST /auth/refresh
- POST /auth/logout
- GET /auth/me

Guards: JwtAuthGuard, RolesGuard

users/

- GET /users/profile
- PATCH /users/profile
- DELETE /users/account (GDPR/DPDP compliance)
- GET /users/notifications
- PATCH /users/notifications/read

lawyers/

- POST /lawyers/register (multi-step, with file upload)
- GET /lawyers/search (Meilisearch-powered)
- GET /lawyers/:id (public profile)
- PATCH /lawyers/profile (authenticated)
- GET /lawyers/:id/availability
- GET /lawyers/:id/reviews

Admin: POST /lawyers/:id/approve, PATCH /lawyers/:id/status

consultations/

- POST /consultations (create booking)
- GET /consultations (user's consultations)
- GET /consultations/:id
- POST /consultations/:id/start
- POST /consultations/:id/end
- POST /consultations/:id/review
- POST /consultations/:id/dispute

messages/ (WebSocket + REST)

- WebSocket: consultation:{id} room
- Events: message:send, message:received, typing, read
- GET /messages/:consultationId (history, paginated)
- POST /messages/:consultationId (REST fallback)

notices/ (Notice Scanner)

- POST /notices/scan (file upload → async job)
- GET /notices/:id (poll result)
- GET /notices (user's scan history)
- POST /notices/:id/save-to-vault

cases/ (Lawyer case management)

- POST /cases
- GET /cases (lawyer's cases, paginated + filtered)
- GET /cases/:id
- PATCH /cases/:id

- DELETE /cases/:id (soft delete)
- POST /cases/:id/hearings
- GET /cases/:id/hearings
- POST /cases/:id/tasks
- PATCH /cases/:id/tasks/:taskId
- POST /cases/:id/documents (file upload)
- GET /cases/:id/documents

documents/ (Vault)

- POST /documents (upload encrypted blob)
- GET /documents (user's vault)
- GET /documents/:id (metadata only)
- DELETE /documents/:id
- POST /documents/:id/share (generate time-limited link)
- GET /documents/shared/:token (access shared doc)

search/ (Meilisearch)

- GET /search/lawyers (lawyer discovery)
- GET /search/judgments (AI research)
- GET /search/content (articles)

ai/

- POST /ai/notice-analyze (internal, called by queue job)
- POST /ai/guide-personalize (emergency guide AI)
- POST /ai/judgment-search (natural language → Meilisearch)
- POST /ai/draft (document drafting)
- POST /ai/document-summarize (vault document insight)

All AI routes: rate-limited, queued, streaming where possible

payments/

- POST /payments/order (create Razorpay order)
- POST /payments/verify (verify signature)
- POST /payments/webhook (Razorpay webhook handler)
- GET /payments/history
- POST /payments/subscription/create
- POST /payments/subscription/cancel

notifications/

- POST /notifications/register-token (Expo push token)
- POST /notifications/send (internal)
- GET /notifications (user's inbox)
- PATCH /notifications/read-all

court-tracker/ (NJDG integration)

- POST /court-tracker/cases (add case to track)
- GET /court-tracker/cases (user's tracked cases)
- GET /court-tracker/cases/:cnr (fetch status from NJDG)
- DELETE /court-tracker/cases/:id

content/ (Articles + Q&A)

- GET /content/articles (paginated, filtered)
- GET /content/articles/:slug
- GET /content/qa (Q&A listing)
- POST /content/qa (ask question – authenticated)
- POST /content/qa/:id/answer (lawyer only)

admin/ (Protected – ADMIN role only)

- GET /admin/lawyers/pending-verification
- POST /admin/lawyers/:id/approve
- POST /admin/lawyers/:id/reject
- GET /admin/users
- GET /admin/consultations
- GET /admin/analytics (platform metrics)
- POST /admin/content/articles (CMS)

BACKGROUND JOBS (Bull Queue):

- notice.scan: OCR + AI analysis (heavy, async)
- email.send: All transactional emails via Resend
- sms.send: OTP + reminders via MSG91
- push.send: Push notifications via Expo
- court.sync: Periodic NJDG status fetch for tracked cases
- search.index: Re-index Meilisearch on data changes
- analytics.aggregate: Nightly stats aggregation

SECURITY REQUIREMENTS:

- All routes protected by JWT unless explicitly public
 - Rate limiting: Auth routes 5/min, API 100/min, AI routes 10/min
 - Input validation: Zod schemas on all request bodies
 - File upload: MIME type validation, size limits (PDF max 10MB, image max 5MB)
 - SQL injection: Drizzle parameterised queries (no raw SQL)
 - XSS: Input sanitisation on all text fields stored as HTML
 - Audit log: All sensitive operations logged (auth, payment, document access)
 - DPDP compliance: Data deletion cascades, data export endpoint, consent tracking
-

10. SHARED PACKAGES BUILD PROMPT {#packages-prompt}

PROMPT: KanooniBaat Shared Packages

Build the shared packages used across web, mobile, and API:

packages/types/ – Shared TypeScript + Zod schemas

Export all entity types matching database schema.

Export Zod schemas for: auth forms, booking forms, case forms, search params.

These schemas are shared between frontend forms and backend validation.

packages/utils/ – Shared utility functions

- formatters: date (Indian format), currency (₹ with commas), phone (Indian +91)
- validators: phone (Indian 10-digit), pincode, PAN, Aadhaar (checksum)
- constants: PRACTICE_AREAS (40+ areas with categories), INDIAN_STATES (all 28+8), COURTS (types + names), LANGUAGES (12 Indian languages + codes)
- i18n keys: all translation key definitions (type-safe)
- legal: IPC to BNS section mapping table (all mapped sections post 2023 laws)

packages/ui/ – Shared UI components (for web, using React + Tailwind)

Note: Mobile has its own components using NativeWind.

Shared: Button, Input, Select, Textarea, Badge, Avatar, Card, Modal, Toast, Skeleton, LoadingSpinner, EmptyState, ErrorBoundary

These are for the web app dashboard sections (not marketing pages).

packages/api-client/ – tRPC client + React Query setup

- tRPC client configured for web + mobile
- All query hooks: useLawyers, useConsultation, useVault, useCases, etc.
- Proper error handling + toast notifications on mutation errors
- Offline support (React Query persistence)

11. AI SERVICES PROMPT {#ai-prompt}

PROMPT: KanooniBaat AI Services Layer

Build the AI orchestration layer using Vercel AI SDK.

MODEL ROUTING:

- Notice analysis: claude-3-5-sonnet (best document understanding)
- Legal research: claude-3-5-sonnet (complex legal reasoning)
- Emergency guide personalisation: claude-3-haiku (fast, cost-efficient)

- Document summarisation: claude-3-haiku (cost-efficient for vault)
- AI drafting: claude-3-5-sonnet (best drafting quality)

NOTICE SCANNER AI PIPELINE:

Input: OCR extracted text (string) + user's preferred language

System prompt: You are an Indian legal expert who explains legal notices in simple, clear language. Always identify the notice type, the real meaning, urgency, key dates, and practical next steps. Never give actual legal advice – always recommend consulting a verified lawyer for specific guidance.

Output (JSON):

```
{
  notice_type: string,
  issuing_authority: string,
  is_likely_genuine: boolean,
  genuineness_flags: string[], // what signals genuine/fake
  plain_summary: string,       // in user's language, max 200 words
  key_dates: { label, date }[],
  key_amounts: { label, amount }[],
  urgency: 'IMMEDIATE' | 'WITHIN_7_DAYS' | 'WITHIN_30_DAYS' | 'NO_DEADLINE',
  recommended_actions: string[], // 3-5 steps
  recommended_lawyer_type: string,
  applicable_laws: string[]
}
```

EMERGENCY GUIDE AI:

Input: scenario type + user's state + context from answers

Output: Personalised guide adapting the base content to the specific state laws and situation details.

LEGAL RESEARCH AI:

Input: natural language query

Output: Meilisearch query params + key search terms + recommended filters

DRAFTING AI:

Input: template type + case facts (structured form)

System: Expert Indian advocate drafting formal legal documents. Use correct court-specific formats. Reference correct BNS/IPC/CPC sections. Output in professional legal English with Hindi option.

Output: Formatted draft document (Markdown → converted to DOCX)

GUARDRAILS (All AI pipelines):

- Never output specific legal advice ("you should do X in your case")
- Always include: "This is general information. Consult a verified KanooniBaat lawyer for advice specific to your situation."

- Never output emergency/medical guidance as legal guidance
- Content filtering: block any output that could be used to evade law enforcement
- Language: always respond in user's preferred language if possible

12. INFRASTRUCTURE & DEVOPS {#infra}

Environment Variables (.env.example)

```
bash
```

Database

```
DATABASE_URL=postgresql://...
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_KEY=...
```

Auth

```
JWT_SECRET=...
JWT_REFRESH_SECRET=...
BETTER_AUTH_SECRET=...
```

SMS

```
MSG91_API_KEY=...
MSG91_SENDER_ID=KANBAT
MSG91_TEMPLATE_ID_OTP=...
```

Email

```
RESEND_API_KEY=...
FROM_EMAIL=hello@kanooni.baat
```

Payments

```
RAZORPAY_KEY_ID=...
RAZORPAY_KEY_SECRET=...
RAZORPAY_WEBHOOK_SECRET=...
```

Storage

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=ap-south-1
AWS_S3_BUCKET=kanooni-baat-docs
CLOUDFRONT_DOMAIN=...
```

AI

```
ANTHROPIC_API_KEY=...
OPENAI_API_KEY=... # fallback
```

OCR

```
GOOGLE_CLOUD_VISION_API_KEY=...
```

Search

```
MEILISEARCH_URL=...
MEILISEARCH_MASTER_KEY=...
```

```
# Cache
UPSTASH_REDIS_URL=...
UPSTASH_REDIS_TOKEN=...

# Video Calls
LIVEKIT_URL=wss://...
LIVEKIT_API_KEY=...
LIVEKIT_API_SECRET=...

# Monitoring
SENTRY_DSN=...
SENTRY_AUTH_TOKEN=...

# Push Notifications
EXPO_ACCESS_TOKEN=...

# WhatsApp (Phase 15)
WHATSAPP_API_TOKEN=...
WHATSAPP_PHONE_NUMBER_ID=...

# App
NEXT_PUBLIC_API_URL=https://api.kanooni.baat
NEXT_PUBLIC_APP_URL=https://www.kanooni.baat
NODE_ENV=production
```

GitHub Actions CI/CD (.github/workflows/ci.yml)

yaml

```
name: CI
on: [push, pull_request]
jobs:
  lint-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
      - run: pnpm install --frozen-lockfile
      - run: pnpm turbo lint
      - run: pnpm turbo typecheck

  test:
    runs-on: ubuntu-latest
    steps:
      - run: pnpm turbo test

  build:
    runs-on: ubuntu-latest
    steps:
      - run: pnpm turbo build

  deploy-web:
    if: github.ref == 'refs/heads/main'
    needs: [lint-typecheck, build]
    runs-on: ubuntu-latest
    steps:
      - run: vercel --prod

  deploy-api:
    if: github.ref == 'refs/heads/main'
    needs: [lint-typecheck, build]
    runs-on: ubuntu-latest
    steps:
      - run: railway up
```

Deployment Architecture

```
Production:
  Web:    Vercel (Edge Runtime, global CDN, ISR)
  API:    Railway (auto-scaling, Mumbai region)
  DB:     Supabase (managed PG, ap-south-1)
  Redis:  Upstash (serverless Redis, global)
```

Search: Meilisearch Cloud (or self-hosted on Railway)

Files: AWS S3 ap-south-1 + CloudFront

Video: LiveKit Cloud (or self-hosted)

Staging:

Web: Vercel preview URLs

API: Railway staging environment

DB: Supabase staging project

PHASE 0: INITIALIZATION PROMPT

PROMPT: Initialize KanooniBaat Monorepo

Create a production-grade Turborepo monorepo for KanooniBaat with the following:

1. ROOT SETUP:

- pnpm workspace (pnpm-workspace.yaml)
- turbo.json with pipelines: build, dev, lint, typecheck, test
- Root package.json with workspace scripts
- .gitignore (node_modules, .turbo, .next, dist, .env)
- .env.example with all required variables (see spec)

2. apps/web – Next.js 15:

- next.config.ts (App Router, Turbopack, image domains)
- TypeScript strict config extending packages/config/typescript
- Tailwind config extending packages/config/tailwind
- src/app directory with layout.tsx, not-found.tsx
- next-intl setup (en + hi)
- Fonts: Geist + Noto Sans Devanagari (next/font)

3. apps/api – NestJS 11:

- Standard NestJS CLI structure
- TypeScript strict
- Drizzle + Supabase setup
- Better Auth integration
- Swagger setup at /api/docs
- Pino logger
- Health check endpoint

4. apps/mobile – Expo SDK 52:

- Expo Router v4

- NativeWind v4
- app.json with correct bundle IDs (com.kanooni.baat)
- EAS build configuration (eas.json)
- Expo fonts setup

5. packages/config:

- typescript/base.json (strict, paths)
- eslint/base.js (+ React, Next.js, React Native rules)
- tailwind/index.ts (shared tokens: colors, fonts, spacing)
- prettier/index.js

6. packages/types:

- All entity TypeScript types + Zod schemas
- Exported with proper barrel exports

7. packages/utils:

- All constants (practice areas, states, courts, languages)
- All formatters and validators
- IPC → BNS section mapping

8. packages/ui:

- Base components with Tailwind + shadcn/ui
- Properly exported for consumption in web app

9. packages/api-client:

- tRPC setup
- React Query configuration
- Base hooks structure

Ensure:

- All packages reference each other correctly (@kb/types, @kb/ui, etc.)
- Turborepo remote caching configured
- pnpm install runs successfully from root
- pnpm dev starts all apps concurrently
- All TypeScript compiles with zero errors
- ESLint passes on all packages

This document is the complete technical and product specification for KanooniBaat. Execute phase by phase. Every phase builds on the previous. Start with Phase 0 (initialization), then Phase 1 (website), then Phase 2 (auth), and so on.

Last updated: 2026 / Version 1.0

